

Essay Reflection

In this activity, I learned how Class works in Object-Oriented Programming in Python. Using Magic: The Gathering cards made it easier for me to understand how classes, objects, and attributes work together. I learned that each object can have its own values even though they are created from the same class.

Instance independence is crucial because each object should maintain its own attribute values. Example scenario is when I have two MTG cards, and I'm assigning the power of one card then it should not change the power of the other card. In a Magic: The Gathering match, this is very important because different cards can have different power, toughness, abilities, mana, or other information in the card. If the instances of two cards were linked, changing the power or any condition of one card could also change the other card. This could cause incorrect calculations and serious gameplay bugs.

I also learned about the self parameter in the `_init_()` method. The self parameter refers to the specific object that was created or used. This allows Python to know what object's attributes should be changed. For example, I used `self.card_name` to store the name of my card. Python requires self to be written clearly because it makes the connection between the object and its attributes clear.

Also, I learned the difference between class attributes and instance attributes. A class attribute, like I used `MagicCard.DEFAULT_SET`, is shared by all instances and if I change it directly through the class, all cards that use the class attribute will be changed to the new value. But, if I write `card1.DEFAULT_SET = "Zendikar"`, Python creates an instance attribute specifically for card1. Shadowing impacts the code because the instance's attribute takes priority over the class attribute and it can be useful when one object needs a different value.